

bambamR: End-to-End RNA-Seq Processing from FASTQ to Publication-Ready Plots

Raban Heller, Hanno Witte, Konrad Steinestel — BwKrhs Ulm

2026-03-31

Contents

1	Introduction	3
1.1	Key Design Principles	3
1.2	Minimal vs. Full Mode	3
2	Installation	3
3	Bundled Example Data	5
4	Normalization	6
4.1	CPM (Counts Per Million)	6
4.2	TPM (Transcripts Per Million)	6
4.3	TMM and RLE (Bioconductor)	6
5	Visualization	7
5.1	PCA Plot	7
5.2	Heatmap	9
5.3	Volcano Plot	10
5.4	MA Plot	11
5.5	Heatmap of Top DE Genes	13
6	Oncoplot	14
6.1	Data Format	14
6.2	Basic Oncoplot	14
6.3	Oncoplot with Clinical Annotations	15
6.4	Selecting Specific Genes	15
6.5	Custom Colors	16
7	FASTQ Import	18

8	Quality Control	18
9	Differential Expression	20
10	Export	20
11	Full Pipeline	21
12	Interactive Shiny App	21
13	Function Reference	22
14	Citation	22
15	Session Info	23

1 Introduction

bambamR is an R package for end-to-end RNA-seq analysis. It covers the full pipeline from raw FASTQ/BAM files through quality control, alignment, read counting, normalization, differential expression analysis, and publication-ready visualizations including onco plots, volcano plots, heatmaps, PCA, and MA plots.

1.1 Key Design Principles

- **Graceful degradation:** All Bioconductor dependencies are optional. The package operates in *minimal mode* (CRAN-only) or *full mode* (with Bioconductor). It never breaks if Bioconductor is missing.
- **Consistent interface:** All exported functions use the `bb_` prefix. All plot functions return `ggplot2` objects for easy customization.
- **Bundled example data:** Every feature can be explored without downloading external files or installing Bioconductor.

1.2 Minimal vs. Full Mode

Feature	Minimal (CRAN)	Full (+ Bioconductor)
CPM / TPM normalization	✓	✓
TMM / RLE normalization	–	✓
DESeq2 / edgeR / limma	–	✓
PCA, volcano, heatmap, MA	✓	✓
Onco plots	✓	✓
FASTQ / BAM import	Base-R	ShortRead / Rsamtools
Alignment wrappers	✓	✓
Read counting	featureCounts CLI	GenomicAlignments
Shiny interactive app	✓	✓

Table 1: Feature availability by operating mode.

2 Installation

Install from CRAN (when available):

```
install.packages("bambamR")
```

Install the development version from GitHub:

```
pak::pak("rabanheller/bambamR")
```

Optionally install Bioconductor packages for full-mode features:

```
BiocManager::install(c(
  "DESeq2", "edgeR", "limma",
  "ShortRead", "Rsamtools",
  "GenomicAlignments", "GenomicRanges"
))
```

```
library(bambamR)
```

3 Bundled Example Data

bambamR ships with ready-to-use datasets accessible via convenience functions:

Function	Contents
<code>bb_example_counts()</code>	200 genes $\times$ 10 samples + metadata
<code>bb_example_mutations()</code>	300 mutations, 50 samples, 20 genes + clinical data
<code>bb_example_de()</code>	500-gene pre-computed DE results

Table 2: Bundled example datasets.

```
# RNA-seq count matrix (200 genes x 10 samples)
ex <- bb_example_counts()
counts <- ex$counts
metadata <- ex$metadata
```

```
dim(counts)
#> [1] 200 10
counts[1:5, 1:5]
#>      Sample_01 Sample_02 Sample_03 Sample_04 Sample_05
#> TP53          525      533      485      515      519
#> KRAS          502      471      498      510      475
#> EGFR          479      504      488      514      473
#> BRAF          454      456      481      453      497
#> PIK3CA        470      469      506      483      532
```

```
metadata
#>      condition batch sex age
#> Sample_01  control    A  M  45
#> Sample_02  control    B  F  52
#> Sample_03  control    A  M  38
#> Sample_04  control    B  F  61
#> Sample_05  control    A  M  55
#> Sample_06 treatment    B  F  47
#> Sample_07 treatment    A  M  59
#> Sample_08 treatment    B  F  42
#> Sample_09 treatment    A  M  50
#> Sample_10 treatment    B  F  44
```

4 Normalization

4.1 CPM (Counts Per Million)

CPM normalization works without any optional dependencies:

```
cpm <- bb_normalize(counts, method = "cpm")
```

4.2 TPM (Transcripts Per Million)

TPM requires gene lengths:

```
gene_lengths <- readRDS(  
  system.file("extdata", "example_gene_lengths.rds", package = "bambamR")  
)  
tpm <- bb_normalize(counts, method = "tpm",  
  gene_lengths = gene_lengths$length)
```

4.3 TMM and RLE (Bioconductor)

With Bioconductor installed:

```
tmm <- bb_normalize(counts, method = "tmm") # requires edgeR  
rle <- bb_normalize(counts, method = "rle") # requires DESeq2
```

5 Visualization

All plot functions return `ggplot2` objects that can be further customized with `+ theme()`, `+ labs()`, etc.

5.1 PCA Plot

```
bb_pca(cpm, metadata, color_by = "condition")
```

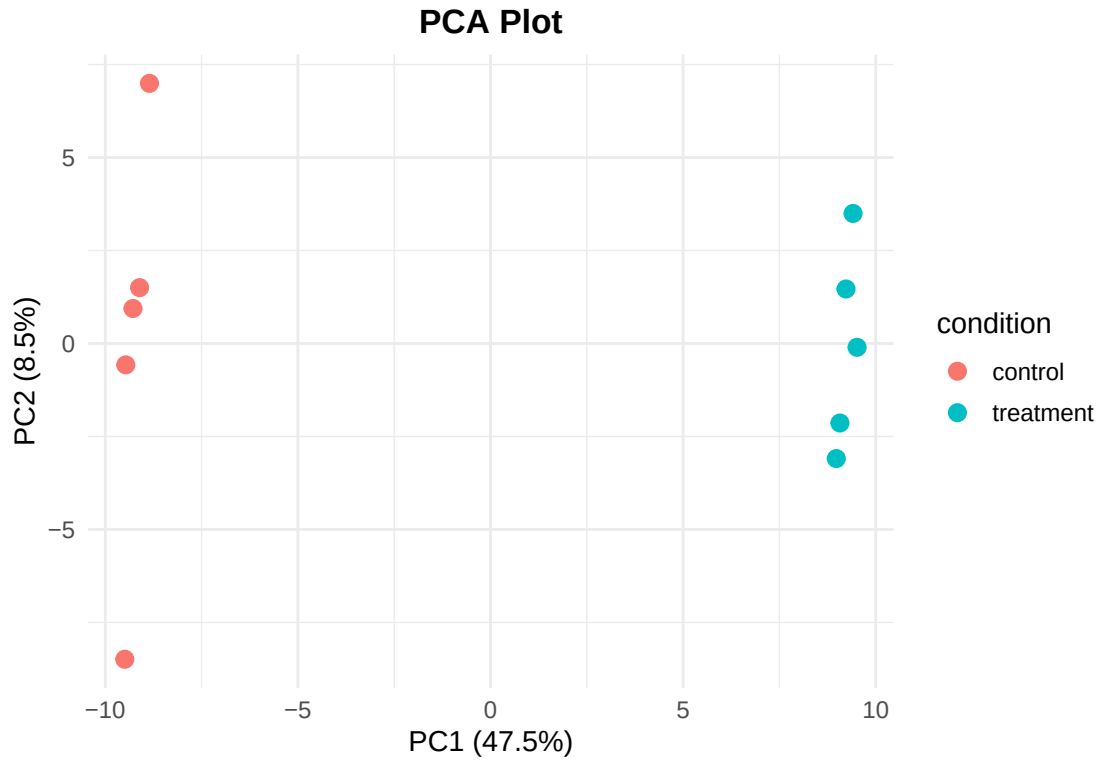


Figure 1: PCA plot colored by experimental condition.

```
bb_pca(cpm, metadata, color_by = "condition",  
       shape_by = "batch", label = TRUE, point_size = 4)
```

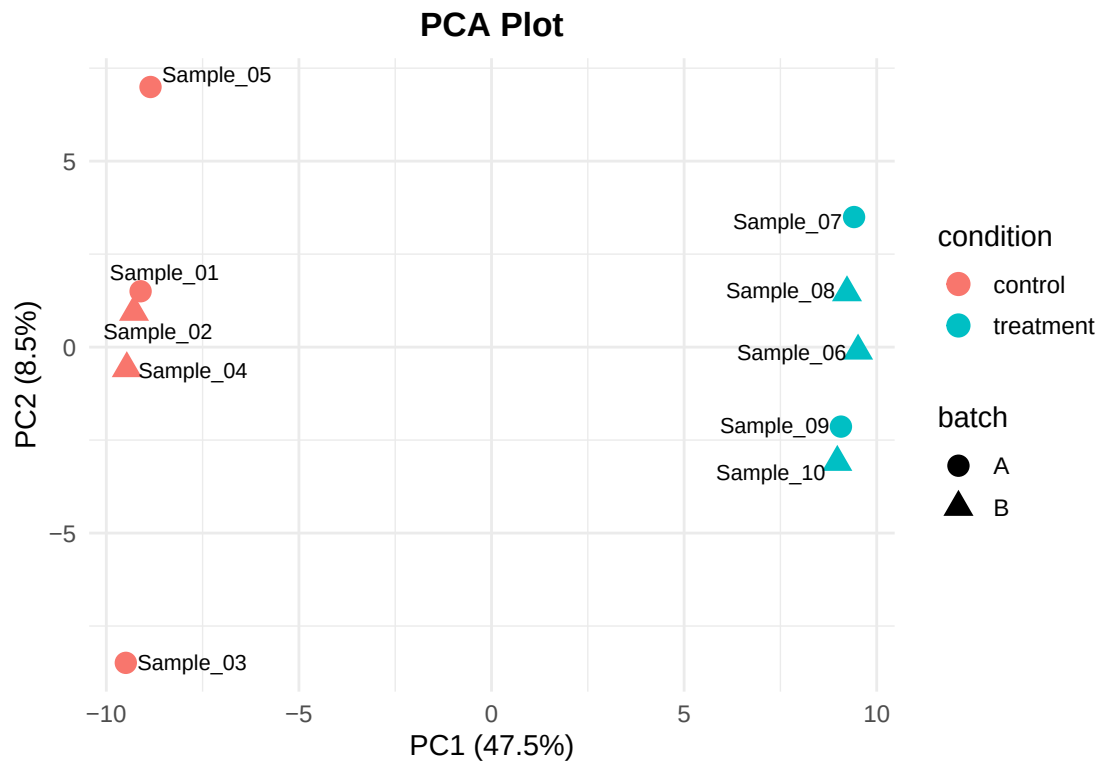


Figure 2: PCA plot with shape mapping for batch effect.

5.2 Heatmap

```
bb_heatmap(cpm, n_genes = 30)
```

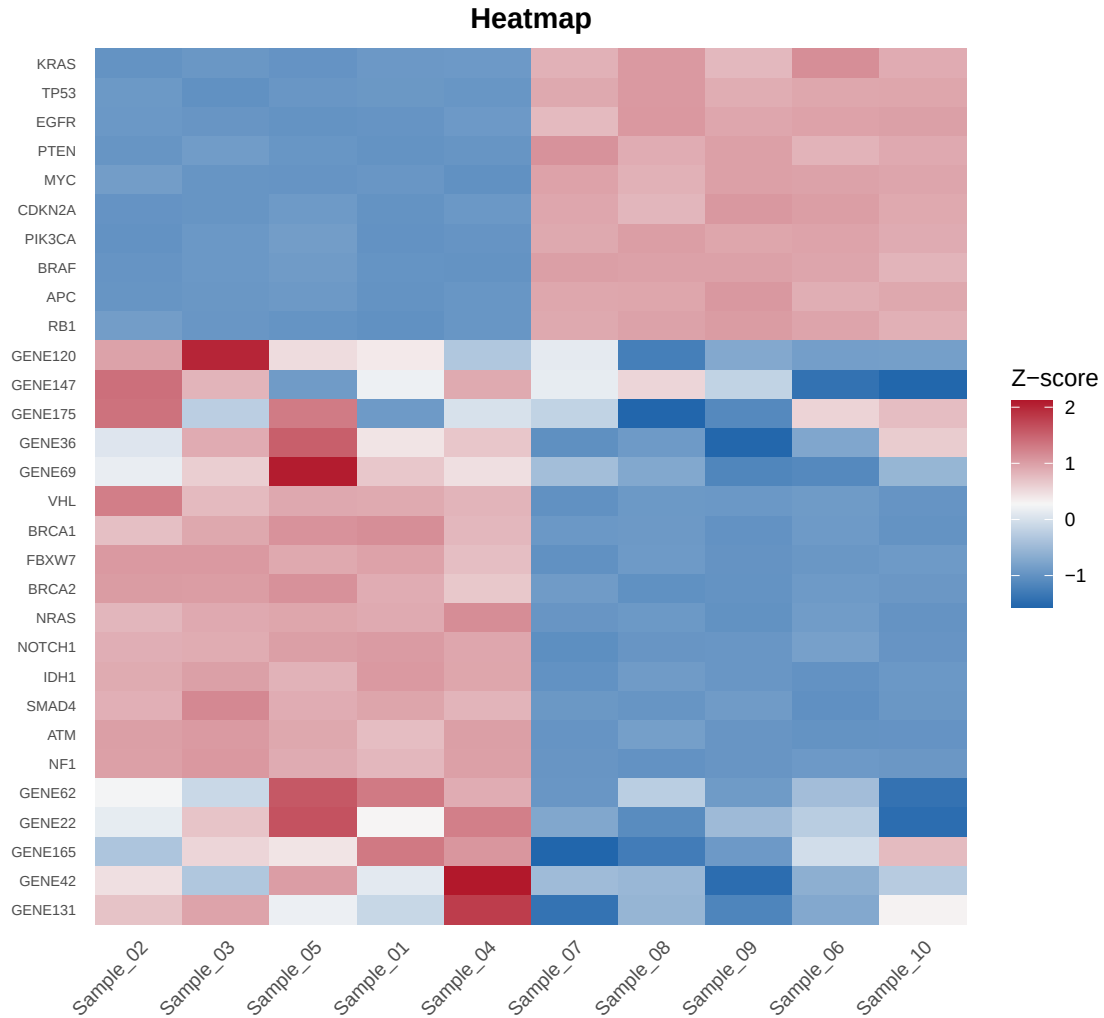


Figure 3: Heatmap of the 30 most variable genes.

5.3 Volcano Plot

Using the pre-computed DE results (no Bioconductor required):

```
de_results <- bb_example_de()
sig <- sum(de_results$padj < 0.05, na.rm = TRUE)
cat("Significant genes (FDR < 0.05):", sig, "\n")
#> Significant genes (FDR < 0.05): 20
```

```
bb_volcano(de_results, fc_cutoff = 1, p_cutoff = 0.05, n_label = 8)
```

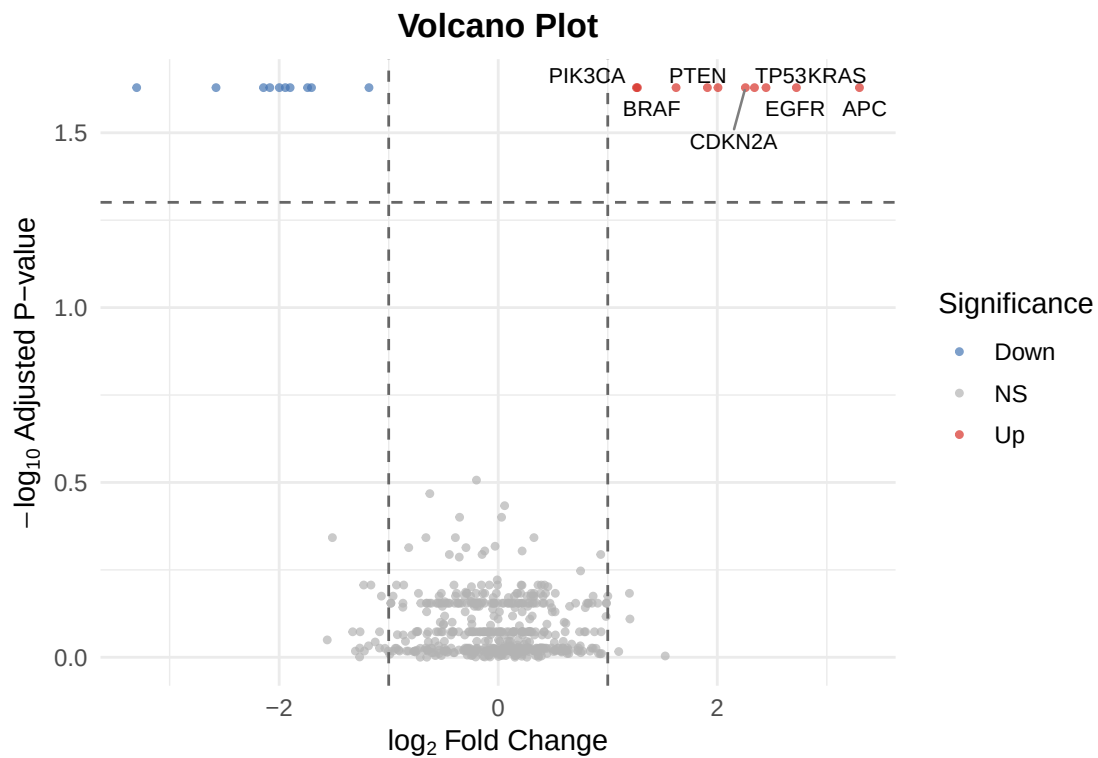


Figure 4: Volcano plot highlighting differentially expressed genes.

```
bb_volcano(de_results,
  fc_cutoff = 0.5,
  p_cutoff = 0.01,
  label_genes = c("TP53", "KRAS", "EGFR", "PTEN"),
  colors = c(up = "#E41A1C", down = "#377EB8", ns = "grey80"))
```

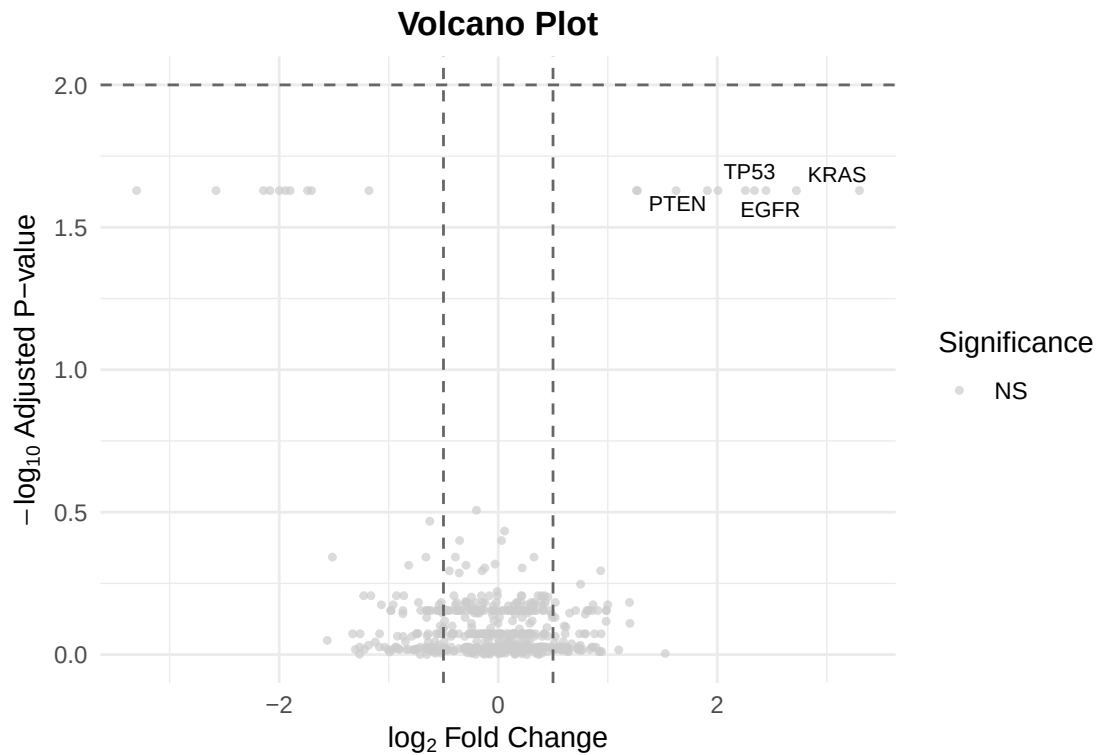


Figure 5: Volcano plot with custom gene labels and colors.

5.4 MA Plot

```
bb_ma_plot(de_results, p_cutoff = 0.05)
```



Figure 6: MA plot showing fold-change vs. mean expression.

5.5 Heatmap of Top DE Genes

```
bb_heatmap(cpm, de_result = de_results, n_genes = 25)
```

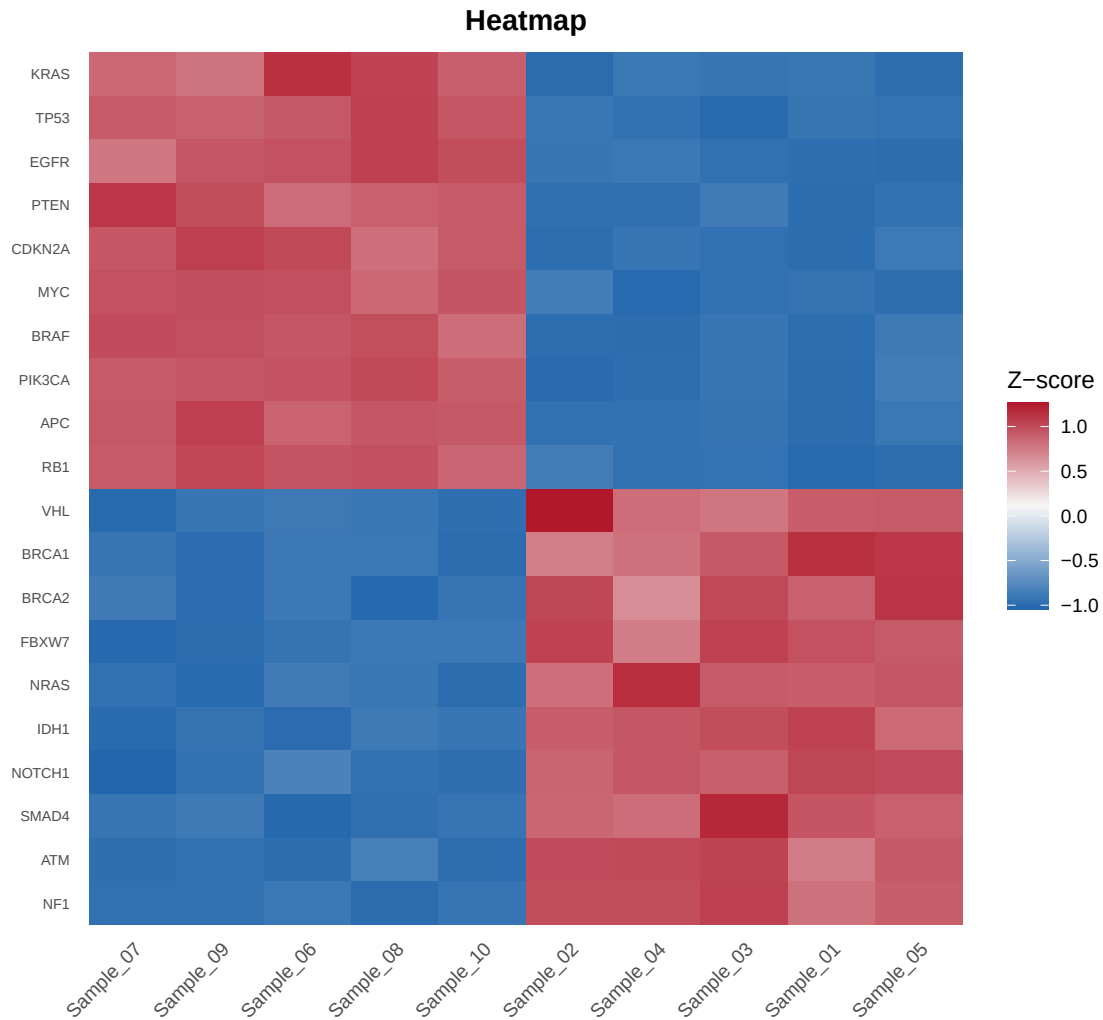


Figure 7: Heatmap of the top 25 differentially expressed genes.

6 Oncoplot

The `bb_oncoplot()` function creates publication-ready waterfall-style mutation landscape plots.

6.1 Data Format

bambamR accepts a `data.frame` with columns `sample`, `gene`, and `mutation_type`, or MAF-format data with `Hugo_Symbol`, `Tumor_Sample_Barcode`, and `Variant_Classification`.

```
mut <- bb_example_mutations()
head(mut$mutations)
#>      sample  gene      mutation_type
#> 1 TCGA-021  NRAS      In_Frame_Ins
#> 2 TCGA-037  PIK3CA  Missense_Mutation
#> 3 TCGA-037   BRAF      Splice_Site
#> 4 TCGA-031  PIK3CA  Frame_Shift_Ins
#> 5 TCGA-042   RB1  Missense_Mutation
#> 6 TCGA-008  NRAS  Missense_Mutation
```

6.2 Basic Oncoplot

```
bb_oncoplot(mut$mutations, n_genes = 10, show_barplot = FALSE)
```

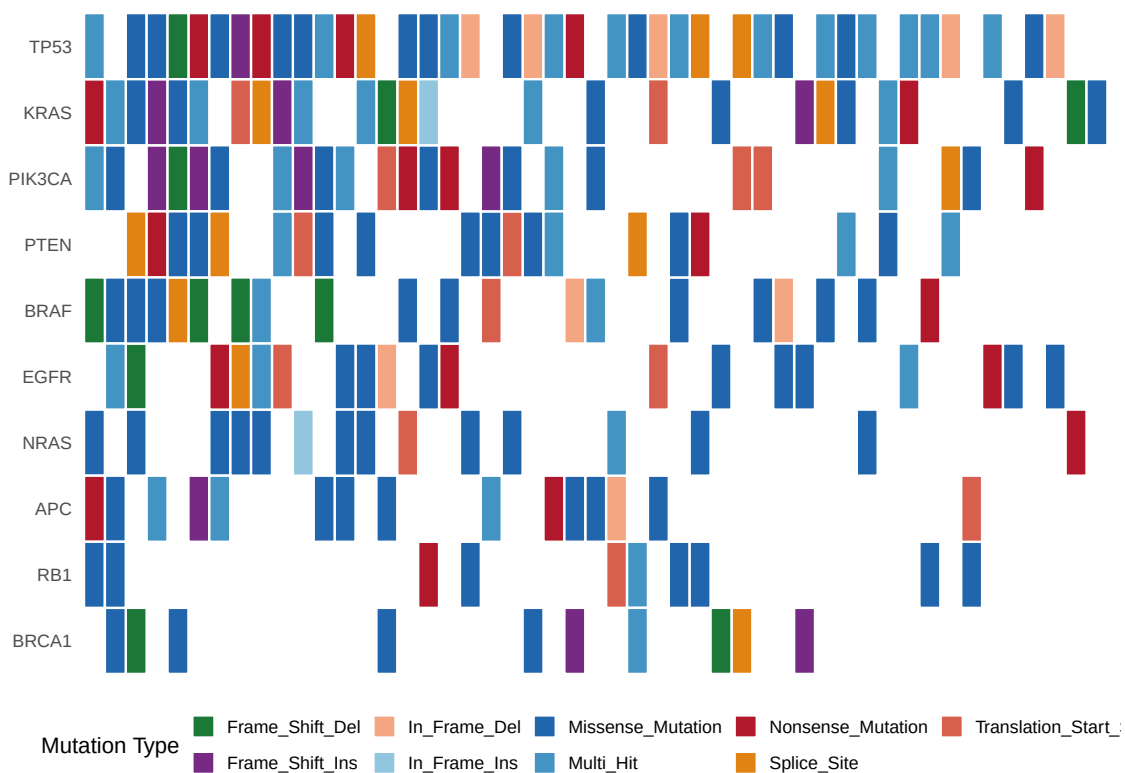


Figure 8: Oncoplot showing the top 10 mutated genes.

6.3 Oncoplot with Clinical Annotations

```
bb_oncoplot(mut$mutations, n_genes = 8,
            annotation_df = mut$clinical, show_barplot = FALSE)
```

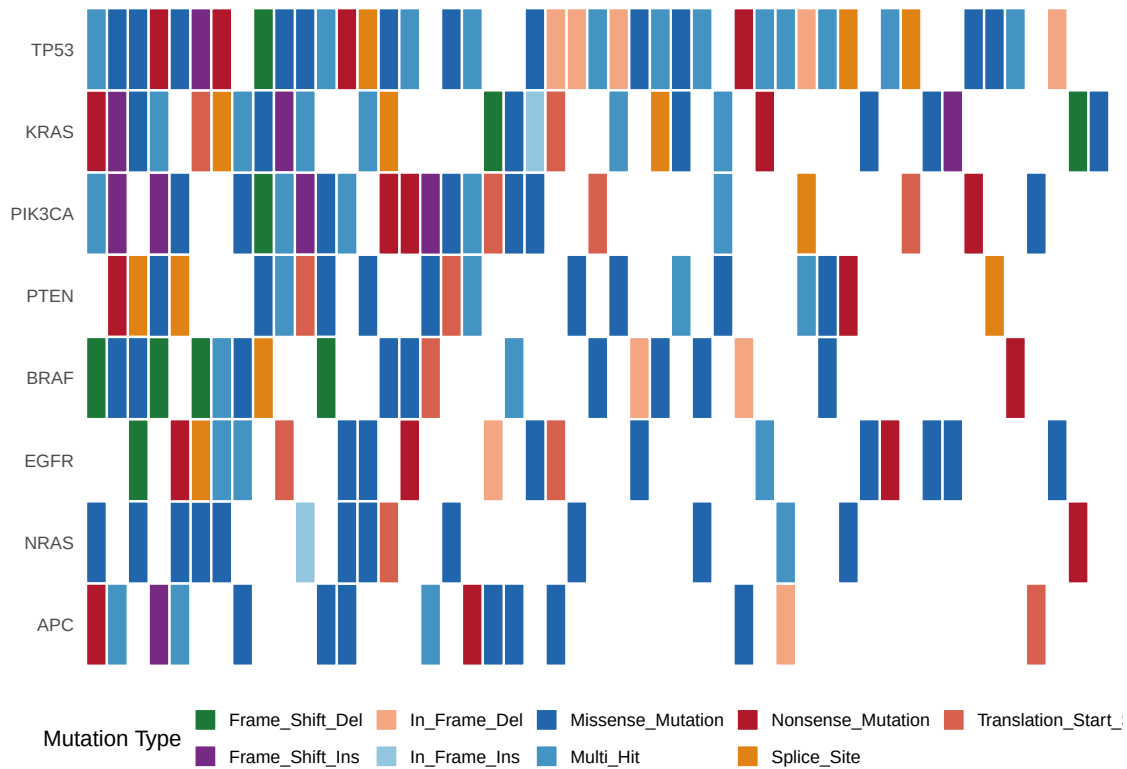


Figure 9: Oncoplot with clinical annotation tracks (Stage, Gender, Smoking).

6.4 Selecting Specific Genes

```
bb_oncoplot(mut$mutations, show_barplot = FALSE,
            genes = c("TP53", "KRAS", "PIK3CA", "BRAF", "EGFR"))
```

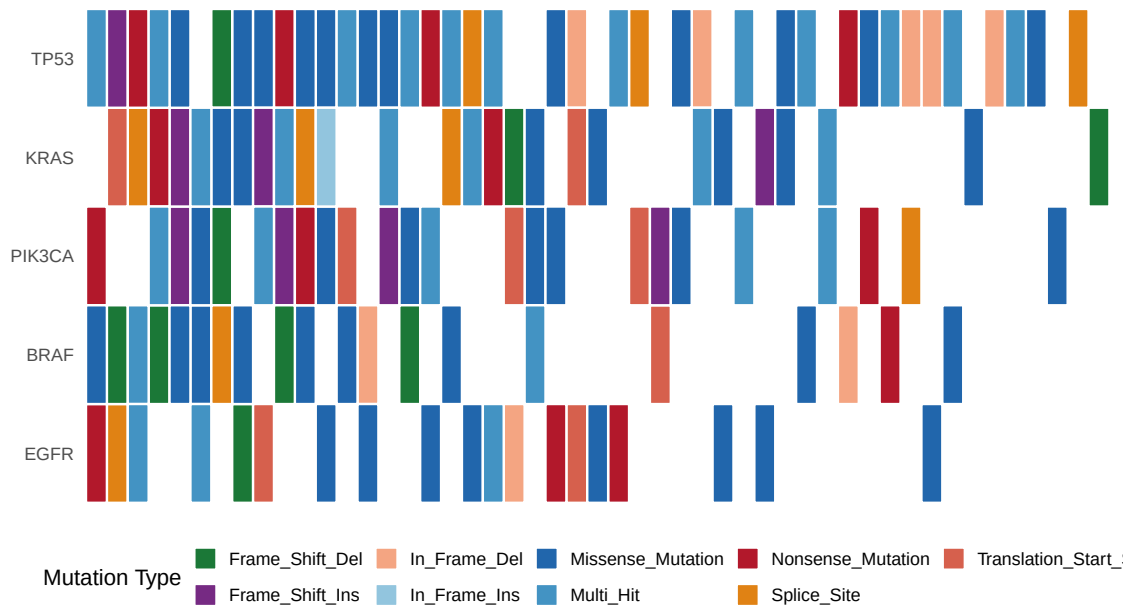


Figure 10: Oncoplot showing five selected genes.

6.5 Custom Colors

```
my_colors <- c(
  "Missense_Mutation" = "#3182BD",
  "Nonsense_Mutation" = "#E6550D",
  "Frame_Shift_Del"   = "#31A354",
  "Frame_Shift_Ins"   = "#756BB1",
  "Splice_Site"       = "#DE2D26",
  "In_Frame_Del"      = "#636363",
  "In_Frame_Ins"      = "#FDAE6B",
  "Translation_Start_Site" = "#BCBDDC",
  "Multi_Hit"         = "#FDD0A2",
  "Other"             = "#BDBDBD"
)
bb_oncoplot(mut$mutations, n_genes = 8,
  mutation_colors = my_colors, show_barplot = FALSE)
```

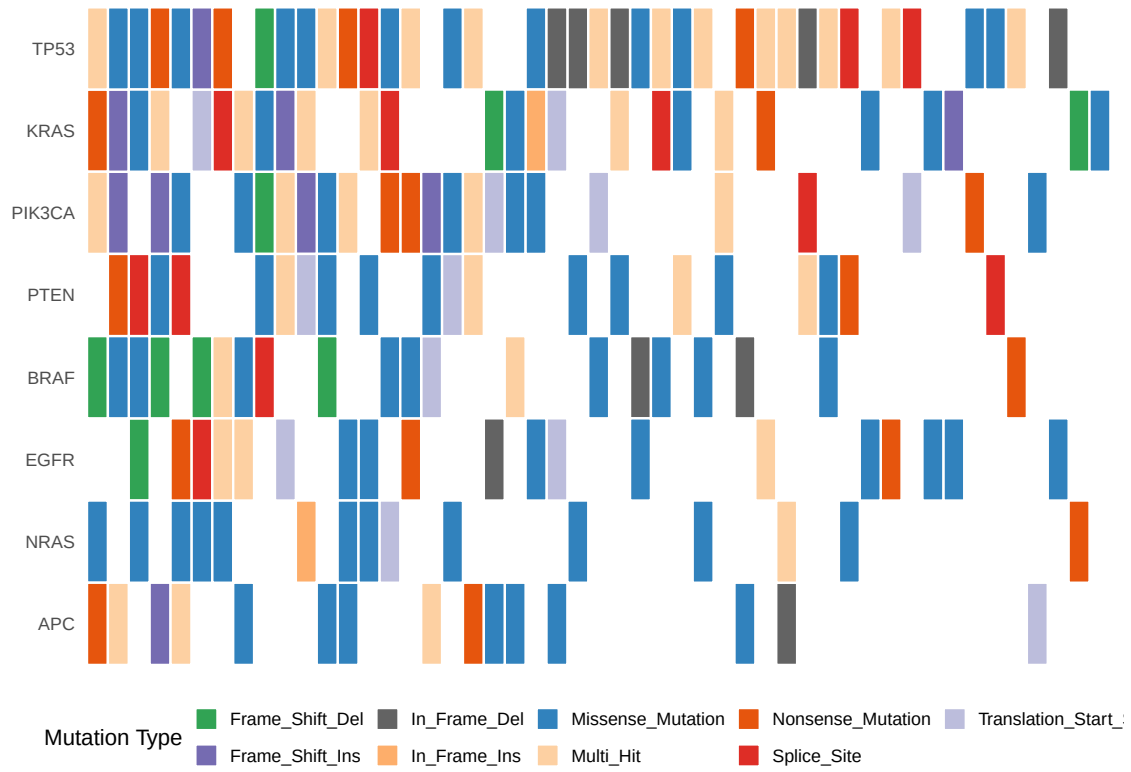


Figure 11: Oncoplot with a custom color palette.

7 FASTQ Import

bambamR includes a small example FASTQ file (100 reads, 75 bp):

```
fq_path <- system.file("extdata", "example_reads.fastq",
                        package = "bambamR")
reads <- bb_read_fastq(fq_path, n = 5)
reads
#>                                id
#> 1 read_0001 length=75
#> 2 read_0002 length=75
#> 3 read_0003 length=75
#> 4 read_0004 length=75
#> 5 read_0005 length=75
#>                                sequence
#> 1 CCCTAGATGAGTGGATTACCTATCGGCGGTATATGTTTCGAAATCTGAGACGCAAAGACCTGTATAAAATTCCC
#> 2 GCGAAGGGATAATAGTCCCTACGAATCTGAAAATGACTAACATGCGTTAGCAACGGTACTGATGGGTATAGGTCA
#> 3 AGGAGTGTTTAACCGGTGACAGCGCCTAATTCTGGAGGGTGGTACCACACACCAACATTACAGATCCGACGACAT
#> 4 ACTAGTCAGGAATCATGTGTGTGATTCCATAGACACATGGCGCGGAAGCCGGCCATATCCCCGTAACGAGCCGCC
#> 5 CGTTCGGCAGATCTACCGGTGTGACTTGACCGGCATTGCTCACCAGAAAGAGTGTAGTTGCCTGAGCCTGTAATG
#>                                quality
#> 1 >FEGGCINGFDIEEIHGIIEFFDCEFGCBEAIIIEGF=IDEEHEFBGAGCIBICIBHCDIHHEGHGA?GEGCCG
#> 2 HGIBCGDHGAIEGADGHFFAIIEFIHFBIGHBIFBBDGDFEIHIAFD=HHC=HHIGGFICH=HDIHHHCIEIIB
#> 3 GHEEHHBABGDHDIFGFFEEGGHIGFEDHEIEDFFIHDGHE>AICHHE>GBHECFCH?EHEIGDEEICCEGIHH
#> 4 EE?IGFFIEGFIEIGIHEIHDHGHEEHFGIHHGFGFCFIHHAIEFHFBHHHAHGH I>EIBFHDIHFFIHHBHFE
#> 5 EIGCIEGHGIDHEICGEDHCGEGHEEHIHDHGIHAEFGIEIB=IFDBIBEGFIGI IHAEG?DIGIBIFHFHG
```

When `ShortRead` is available, `bb_read_fastq()` uses it automatically. Otherwise, a base-R parser handles plain and gzipped FASTQ files.

8 Quality Control

```
qc <- bb_qc(fastq_path = fq_path)
qc
#> bambamR QC Summary
#> =====
#> Files analyzed: 1
#> example_reads.fastq: 100 reads
```

```
bb_qc_summary(qc)
#>                                file total_reads median_gc mapping_rate
#> 1 example_reads.fastq                100 0.4933333                NA
```

```
bb_plot_qc(qc)
```

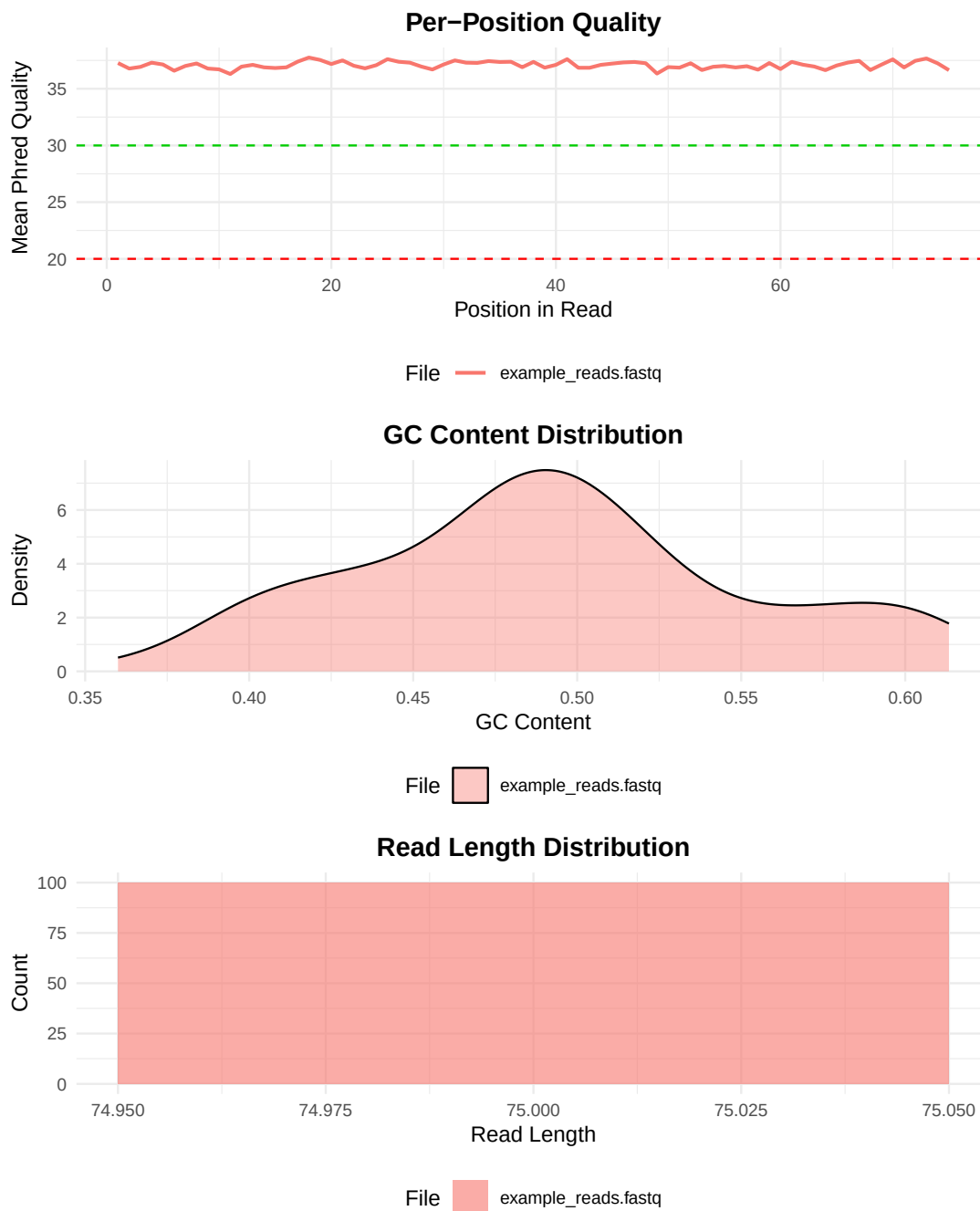


Figure 12: QC visualization panel: per-position quality, GC content, and read length distribution.

9 Differential Expression

Three DE methods are supported, each requiring an optional Bioconductor package. All three return a standardized `data.frame` with columns `gene`, `log2fc`, `pvalue`, `padj`, and optionally `basemean`.

```
# DESeq2 (requires DESeq2)
de <- bb_deseq2(counts, metadata)

# edgeR (requires edgeR)
de <- bb_edger(counts, group = metadata$condition)

# limma-voom (requires limma + edgeR)
design <- model.matrix(~ condition, data = metadata)
de <- bb_limma_voom(counts, design)
```

The standardized output ensures that `bb_volcano()`, `bb_ma_plot()`, `bb_heatmap()`, and `bb_export_csv()` work identically regardless of the DE method used.

10 Export

```
bb_export_csv(de_results, "de_results.csv")
bb_export_tsv(de_results, "de_results.tsv")
bb_export_rds(de_results, "de_results.rds")
```

11 Full Pipeline

The `bb_pipeline()` function runs the entire analysis end-to-end. It accepts entry at three stages:

```
# From FASTQ files
result <- bb_pipeline(
  fastq_dir      = "raw_reads/",
  genome_index   = "STAR_index/",
  annotation      = "genes.gtf",
  sample_info    = metadata,
  aligner        = "STAR",
  de_method       = "DESeq2",
  threads        = 8
)

# From BAM files (skip alignment)
result <- bb_pipeline(
  bam_dir        = "aligned/",
  annotation      = "genes.gtf",
  sample_info    = metadata
)

# From a count matrix (skip alignment + counting)
result <- bb_pipeline(
  count_matrix   = my_counts,
  sample_info    = metadata,
  de_method       = "edgeR"
)
```

The returned `bb_result` object bundles counts, metadata, DE results, and all generated plots:

```
result$counts      # count matrix
result$de_results  # standardized DE data.frame
result$plots$pca   # ggplot object
result$plots$volcano # ggplot object
result$plots$heatmap # ggplot object
```

12 Interactive Shiny App

Launch a browser-based dashboard for interactive analysis:

```
bb_run_app()
```

The app provides:

- Data upload (CSV, TSV, RDS) with live preview
- Configurable normalization and DE parameters
- Interactive plots (volcano, PCA, heatmap, MA, oncoplot)
- One-click export (CSV, RDS, PDF)

13 Function Reference

Function	Description
<i>Import</i>	
<code>bb_read_fastq()</code>	Read FASTQ files
<code>bb_read_bam()</code>	Read BAM files
<code>bb_count_bam()</code>	Count reads in BAM
<i>Quality Control</i>	
<code>bb_qc()</code>	Compute QC metrics
<code>bb_qc_summary()</code>	Tabular QC summary
<code>bb_plot_qc()</code>	QC visualization panel
<i>Alignment & Counting</i>	
<code>bb_align()</code>	Align with STAR / HISAT2 / minimap2
<code>bb_count_reads()</code>	Count reads per gene
<i>Normalization</i>	
<code>bb_normalize()</code>	CPM, TPM, TMM, or RLE
<i>Differential Expression</i>	
<code>bb_deseq2()</code>	DESeq2 wrapper
<code>bb_edger()</code>	edgeR quasi-likelihood wrapper
<code>bb_limma_voom()</code>	limma-voom wrapper
<i>Visualization</i>	
<code>bb_oncoplot()</code>	Waterfall mutation plot
<code>bb_volcano()</code>	Volcano plot
<code>bb_heatmap()</code>	Clustered heatmap
<code>bb_pca()</code>	PCA plot
<code>bb_ma_plot()</code>	MA plot
<i>Pipeline & Export</i>	
<code>bb_pipeline()</code>	Full end-to-end pipeline
<code>bb_export_csv()</code>	Export as CSV
<code>bb_export_tsv()</code>	Export as TSV
<code>bb_export_rds()</code>	Export as RDS
<i>Example Data</i>	
<code>bb_example_counts()</code>	Example count matrix
<code>bb_example_mutations()</code>	Example mutation data
<code>bb_example_de()</code>	Example DE results
<i>App</i>	
<code>bb_run_app()</code>	Launch Shiny dashboard

Table 3: Complete function reference.

14 Citation

If you use bambamR in your research, please cite:

Heller R, Witte H, Steinestel K (2026). *bambamR: End-to-End RNA-Seq Processing from FASTQ to Publication-Ready Plots*. R package version 0.1.0. <https://github.com/rabanheller/bambamR>

15 Session Info

```

sessionInfo()
#> R version 4.5.2 (2025-10-31 ucrt)
#> Platform: x86_64-w64-mingw32/x64
#> Running under: Windows 11 x64 (build 26200)
#>
#> Matrix products: default
#> LAPACK version 3.12.1
#>
#> locale:
#> [1] LC_COLLATE=English_Germany.utf8 LC_CTYPE=English_Germany.utf8
#> [3] LC_MONETARY=English_Germany.utf8 LC_NUMERIC=C
#> [5] LC_TIME=English_Germany.utf8
#>
#> time zone: Europe/Berlin
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods    base
#>
#> other attached packages:
#> [1] bambamR_0.1.0
#>
#> loaded via a namespace (and not attached):
#> [1] tidyselect_1.2.1      dplyr_1.2.0
#> [3] farver_2.1.2          Biostrings_2.78.0
#> [5] S7_0.2.1              bitops_1.0-9
#> [7] fastmap_1.2.0         GenomicAlignments_1.46.0
#> [9] digest_0.6.39         lifecycle_1.0.5
#> [11] pwalign_1.6.0         cluster_2.1.8.2
#> [13] statmod_1.5.1         magrittr_2.0.4
#> [15] compiler_4.5.2        rlang_1.1.7
#> [17] tools_4.5.2           yaml_2.3.12
#> [19] knitr_1.51            S4Arrays_1.10.1
#> [21] labeling_0.4.3        interp_1.1-6
#> [23] DelayedArray_0.36.0   RColorBrewer_1.1-3
#> [25] ShortRead_1.68.0      abind_1.4-8
#> [27] BiocParallel_1.44.0   withr_3.0.2
#> [29] hwriter_1.3.2.1       BiocGenerics_0.56.0
#> [31] grid_4.5.2            stats4_4.5.2
#> [33] latticeExtra_0.6-31   colorspace_2.1-2
#> [35] edgeR_4.8.2           ggplot2_4.0.2
#> [37] scales_1.4.0          iterators_1.0.14
#> [39] SummarizedExperiment_1.40.0 cli_3.6.5
#> [41] rmarkdown_2.31        crayon_1.5.3
#> [43] generics_0.1.4        otel_0.2.0
#> [45] rjson_0.2.23          parallel_4.5.2
#> [47] XVector_0.50.0        matrixStats_1.5.0
#> [49] vctrs_0.7.1           Matrix_1.7-4
#> [51] IRanges_2.44.0        GetoptLong_1.1.0
#> [53] patchwork_1.3.2       S4Vectors_0.48.0
#> [55] ggrepel_0.9.8         clue_0.3-68

```

```
#> [57] jpeg_0.1-11          locfit_1.5-9.12
#> [59] foreach_1.5.2        limma_3.66.0
#> [61] glue_1.8.0           codetools_0.2-20
#> [63] gtable_0.3.6         shape_1.4.6.1
#> [65] deldir_2.0-4         GenomicRanges_1.62.1
#> [67] ComplexHeatmap_2.26.1 tibble_3.3.1
#> [69] pillar_1.11.1        htmltools_0.5.9
#> [71] Seqinfo_1.0.0        circlize_0.4.17
#> [73] R6_2.6.1             doParallel_1.0.17
#> [75] evaluate_1.0.5       lattice_0.22-7
#> [77] Biobase_2.70.0       png_0.1-9
#> [79] Rsamtools_2.26.0     cigarillo_1.0.0
#> [81] Rcpp_1.1.1           SparseArray_1.10.9
#> [83] DESeq2_1.50.2        xfun_0.57
#> [85] MatrixGenerics_1.22.0 pkgconfig_2.0.3
#> [87] GlobalOptions_0.1.3
```